HTWG Konstanz

Titel der Masterarbeit

Exposé zur Masterarbeit

Betreuer-Ausschreibung: *Titel der Ausschreibung*

Joschka Peeters

Matrikelnummer: 301750

15. Juli 2026

Betreuer: Prof. Dr. Rainer Müller

Inhaltsverzeichnis

1 Problemstellung / Motivation **3**

1.1 Das Reaktivitätsproblem 4
1.2 Das Strukturproblem 4
1.3 Das Transparenzproblem 5
1.4 Zielsetzung 5

2 Forschungsstand **6**

2.1 Coding-Agents und der Aufgabenrahmen 6
2.2 Kontext als Erfolgsfaktor und Engpass 8
2.3 Graphbasierte Kontextbereitstellung 8
2.4 Grenzen langer, unstrukturierter Kontexte (Modellseite) 8
2.5 Nachvollziehbarkeit und menschliche Beteiligung 9

3 Abgrenzung **9**

3.1 Einleitung 9
3.2 Strang 1: Automatisches graphbasiertes Code-Retrieval 10
3.3 Strang 2: Interaktive Inspektion von Retrieval-Pipelines 11
3.4 Strang 3: Menschengestützte Interfaces für Wissensgraphen 12
3.5 Abgrenzung und Beitrag 13

1 Problemstellung / Motivation

Ziel dieser Masterarbeit ist die Entwicklung eines Systems, mit dem Softwareentwickler, die einen Coding-Agenten einsetzen, den Kontext, auf dem dieser arbeitet, vor der Ausführung prüfen und anpassen können.

Ein Coding-Agent ist eine auf einem Sprachmodell basierende Softwarekomponente, die Programmieraufgaben weitgehend selbstständig löst. Im Gegensatz zu klassischen Sprachmodellen, die primär eine direkte Antwort auf eine Eingabe generieren, verfolgen agentenbasierte Systeme einen mehrstufigen Lösungsansatz. Sie können Aufgaben selbstständig analysieren, weitere Verarbeitungsschritte planen, externe Werkzeuge verwenden, Befehle ausführen und die daraus gewonnenen Rückmeldungen in ihr weiteres Vorgehen einbeziehen (Xia u. a. 2024). In der Softwareentwicklung werden solche Agenten immer stärker eingesetzt. Vertreter wie OpenAI Codex und Claude Code navigieren durch ein Repository und schlussfolgern über mehrere Dateien hinweg. Sie sind damit nicht mehr bloße Assistenten, sondern eigenständige Beitragende (Lulla u. a. 2026).

Ein etablierter Maßstab dafür, wie gut solche Agenten Programmieraufgaben lösen, ist der SWE-bench-Benchmark: Er stellt Sprachmodelle vor reale Aufgaben aus der Softwareentwicklung, die in Form von GitHub-Issues dokumentiert sind. Das Modell erhält die Beschreibung des Issues und den zugehörigen Codebase-Stand und muss eine gezielte Änderung am Quellcode zur Behebung des Problems erzeugen (Patch), der die hinterlegten Tests besteht (Jimenez u. a. 2024).

Eine reale Codebase ist allerdings zu groß, um vollständig in das Kontextfenster des vom Agenten genutzten Sprachmodells zu passen (Jimenez u. a. 2024). Das Kontextfenster bezeichnet die in Tokens gemessene Textmenge, die ein Sprachmodell gleichzeitig verarbeiten kann (Du u. a. 2025). Wie gut ein Agent eine Aufgabe löst, hängt deshalb davon ab, welchen Ausschnitt er sich als Kontext heraussucht, denn auf diesem Kontext baut sein Schlussfolgern auf. Genau hier liegt aber eine Schwäche aktueller Agenten-Systeme: Sie rufen möglichst breiten Kontext ab, um nichts zu übersehen, beziehen dabei aber viel Rauschen ein und bleiben ungenau (Li u. a. 2026).

Diese Ungenauigkeit hängt somit nicht nur vom bloßen Umfang des abgerufenen Kontextes ab, sondern maßgeblich auch von der Art und Weise seiner Bereitstellung. Aus dieser ungesteuerten und rein automatischen Form der Bereitstellung ergeben sich drei Probleme, die in den folgenden Abschnitten einzeln behandelt werden. Der Abruf relevanter Code-Fragmente geschieht reaktiv während der Exploration durch den Agenten, indem er schrittweise auf Werkzeugausgaben und Fehlermeldungen reagiert, statt vorab zielgerichtet gesteuert zu werden (Reaktivitätsproblem). Um für die Bearbeitung einer Fragestellung aus dem zu großen Gesamtkontext die relevanten Teile auszuwählen, erstel-

len fortgeschrittene Ansätze einen Wissensgraphen, dessen struktureller Vorteil für die Kontextauswahl in Abschnitt 2.3 näher betrachtet wird. Dieser bildet die Code-Einheiten als Knoten und ihre Abhängigkeiten als Kanten ab und gewichtet algorithmisch, welche Teile für die Fragestellung relevant sind. Das Modell erhält anschließend die höher gewichteten Teile als Kontext. Dabei geht die im Graphen angelegte Beziehungsstruktur bei der Übergabe an das Modell teilweise wieder verloren, da sie zu flachen Listen komprimiert wird (Strukturproblem). Zudem bleibt für den Entwickler völlig offen, wie die zugrundeliegende Gewichtung und die daraus resultierende Auswahl genau zustande kommen (Transparenzproblem).

1.1 Das Reaktivitätsproblem

Wie aktuelle Coding-Agents ihren Kontext beschaffen, haben Li u. a. (2026) an Modellen wie GPT-5 und Claude Sonnet 4.5 untersucht. Dazu zeichnen sie die gesamte Trajektorie eines Agenten, also den vollständigen Ablauf seiner Schritte, auf und halten jede Code-Stelle fest, die er während der Bearbeitung reaktiv sichtet. Über alle untersuchten Modelle hinweg zeigt sich dasselbe Muster: Die Agenten rufen Code-Fragmente in einem zu großen Umfang ab und setzen auf Vollständigkeit statt auf Genauigkeit. Dies führt zu einem massiven Verbrauch an Token, wobei ein erheblicher Teil der abgerufenen Code-Fragmente letztlich überhaupt nicht zur Lösung beiträgt. Dass eine ungenaue Aufgabenstellung die Suche zusätzlich ausufern lässt, zeigen Suri u. a. (2026): In ihrem einführenden Beispiel durchsucht ein Agent ein Django-Repository 21 Schritte lang, ohne den Fehler zu beheben, während dieselbe Aufgabe mit einer präzisierten Beschreibung in sechs Schritten gelingt.

Verschiedene Ansätze bereiten den Kontext bereits vorab auf. *Agentless* (Xia u. a. 2024) grenzt die Fehlerstelle schrittweise ein, von der Datei über die Funktion bis zur einzelnen Zeile. *CodeScout* (Suri u. a. 2026) ergänzt die Aufgabenbeschreibung vorab um Hinweise aus dem Repository. Beide Ansätze laufen jedoch vollautomatisch ab: Welche spezifischen Codeabschnitte und technischen Details letztendlich an das Sprachmodell übergeben werden, entscheidet das System selbst und nicht der anwendende Entwickler.

1.2 Das Strukturproblem

Um die enorme Größe einer Datenbasis strukturiert zu bewältigen und präzise, nachvollziehbare Antworten zu ermöglichen, bieten Wissensgraphen (Knowledge Graphs) eine etablierte Alternative zu unstrukturiertem Text. Doch auch ein gut ausgewählter Kontext kann in einer strukturell ungünstigen Form beim Agenten ankommen. Pusch u. a. (2026)

beschreiben für wissensgraphenbasierte Retrieval-Augmented-Generation-Pipelines (KG-RAG), also Verfahren, die einem Sprachmodell vor der Antwortgenerierung passende Informationen aus einem Wissensgraphen abrufen und in den Kontext geben, das Problem, dass manche Systeme den Graphen vor der Übergabe in eine bloße, flache Liste von Subjekt-Prädikat-Objekt-Tripeln zerlegen, also in einzelne Aussagen, die je zwei Knoten über eine Beziehung verbinden, und diese an das Sprachmodell übergeben. Der resultierende Text wird dadurch sehr lang und bläht das Kontextfenster unnötig auf. Zudem geht die Beziehungsstruktur, die im Graphen ausdrücklich vorhanden war, verloren, sodass das Modell die relationalen Abhängigkeiten aus der Liste erst mühsam wieder erschließen muss.

1.3 Das Transparenzproblem

Um den Kontext eines Agenten mitgestalten zu können, muss man zuerst sehen, wie er zustande kommt. Genau hier setzen Pusch u. a. (2026) an und schlagen eine Lösung mit menschlicher Beteiligung vor. Das Sprachmodell formuliert die Anfrage an den Graphen und erklärt sie in verständlicher Sprache, der Nutzer gibt Rückmeldung, und beide verfeinern die Anfrage schrittweise gemeinsam. Der Mensch steuert dabei die Formulierung der Anfrage, nicht die daraus resultierende Auswahl der abgerufenen Knoten.

Für Coding-Agents gibt es inzwischen Werkzeuge, die die Trajektorie eines Agenten für dessen Entwickler aufbereiten, etwa *AgentStepper* (Hutter und Pradel 2026). Das Werkzeug stellt den Ablauf als strukturierte Konversation zwischen Sprachmodell, Agentenprogramm und Werkzeugen dar und zeigt neben den einzelnen Werkzeugaufrufen auch die schrittweisen Code-Änderungen des Agenten an. Sichtbar werden dabei ebenfalls die vom Sprachmodell ausgegebenen Begründungen, also die thoughts des Agenten. Ob diese Selbstauskunft mit dem tatsächlichen Auswahlmechanismus übereinstimmt, geht aus der Darstellung jedoch nicht hervor. Warum eine bestimmte Datei in den Kontext gelangt und eine andere unberücksichtigt bleibt, lässt sich anhand der Trajektorie nur begrenzt beurteilen. Hinzu kommt, dass das Werkzeug in erster Linie Entwickler des Agenten adressiert und nicht Nutzer, die einen Coding-Agent im Arbeitsalltag einsetzen.

1.4 Zielsetzung

Ziel dieser Arbeit ist die Entwicklung eines Systems, das Coding-Agents ihren Kontext über ein graphbasiertes Retrieval, also den gezielten Abruf passender Code-Teile aus dem Graphen, bereitstellt und den Softwareentwickler an jedem Schritt dieser Auswahl beteiligt. Grundlage ist ein Graph der Codebase, in dem Dateien, Funktionen

und ihre Abhängigkeiten als Knoten und Kanten liegen. Das Retrieval-Werkzeug ist über das Model Context Protocol (MCP) angebunden, einen offenen Standard, über den Coding-Agents externe Werkzeuge und Datenquellen einheitlich aufrufen können (Anthropic 2024). Ruft der Agent es auf, öffnet sich eine interaktive Oberfläche. Sie zeigt den abgerufenen Kontext als Graph und legt zu jedem Knoten offen, warum er abgerufen wurde. Das Retrieval läuft dort schrittweise ab. Das System ermittelt passende Knoten und folgt den Verbindungen zu weiterem zusammenhängendem Code. Der Entwickler prüft dabei jeden Schritt, wählt einzelne Knoten ab oder fügt weitere hinzu, ordnet sie nach Wichtigkeit und gibt die Richtung vor, etwa welche Startknoten den Ausgangspunkt bilden, welchen Abhängigkeiten gefolgt wird und wie weit, also über wie viele Hops. Erst wenn er die Auswahl bestätigt, geht der kuratierte Graph samt der vergebenen Priorisierung in strukturierter Form an den Agenten, der auf dieser Grundlage weiterarbeitet. Beim klassischen Retrieval-augmented Generation (RAG) wählt ein Retriever die Textausschnitte automatisch nach Ähnlichkeit aus und fügt sie direkt in den Prompt ein. Hier dagegen entsteht die Auswahl schrittweise und unter Steuerung des Entwicklers.

Damit greift der Ansatz die drei zuvor benannten Punkte auf. Der Entwickler gestaltet die Auswahl der benötigten Kontextbestandteile und ihrer Gewichtung, bevor der Agent damit weiterarbeitet, und muss den Ablauf nicht erst im Nachhinein nachvollziehen. Zu jedem Knoten ist sichtbar, warum er aufgenommen wurde, sodass die Auswahl nachvollziehbar bleibt. Auch die Struktur des Graphen bleibt erhalten und zerfällt nicht zu einer flachen Liste. Genau darüber hilft die Struktur dem Agenten: Das Retrieval erschließt den Kontext als geordnete Traversierung entlang der Verknüpfungen, sodass zusammenhängender Code über die Kanten erreicht wird, statt den gesamten Kontext zu laden, und die vergebene Gewichtung bestimmt dabei, welche Knoten den Agenten zuerst erreichen. Wie sich eine solche menschliche Kuratierung auf die Ergebnisse eines Coding-Agenten auswirkt, prüft die Arbeit in einer kontrollierten Studie, die das kuratierte Retrieval mit einem rein automatischen vergleicht.

2 Forschungsstand

2.1 Coding-Agents und der Aufgabenrahmen

Um Fortschritte beim automatischen Lösen realer Softwareentwicklungsaufgaben vergleichbar zu machen, hat sich SWE-bench als gemeinsamer Maßstab etabliert (Jimenez u. a. 2024). Der Benchmark umfasst 2.294 reale GitHub-Issues aus zwölf populären Python-Repositories, jeweils zusammen mit dem Pull Request, der das Issue gelöst

hat. Ein Issue ist dabei ein im Tracker des Repositories gemeldeter Fehler oder Änderungswunsch. Zu jeder Aufgabe erhält ein Modell die Beschreibung dieses Issues und das zugehörige Repository und muss den Code so ändern, dass er die hinterlegten Tests besteht. Als Lösungsrate wird der Anteil der Aufgaben angegeben, bei denen anschließend alle Tests erfolgreich durchlaufen. Die Aufgaben in SWE-bench verlangen häufig abgestimmte Änderungen über mehrere Funktionen, Klassen und Dateien hinweg sowie den Umgang mit langen Kontexten (Jimenez u. a. 2024).

Yang u. a. (2024) stellen mit *SWE-agent* ein System vor, das dem Sprachmodell Werkzeuge an die Hand gibt und es selbst entscheiden lässt, welche Schritte es geht. Damit ein Modell in der Umgebung des SWE-bench-Benchmarks zurechtkommt, legen sie eine Zwischenschicht zwischen das Modell und seine Arbeitsumgebung, das Agent-Computer Interface. Sie stellt wenige klare Befehle zum Suchen, Öffnen und Bearbeiten von Dateien bereit und meldet nach jedem Befehl knapp zurück, was er bewirkt hat. Welche Vorteile diese Schicht liefert, zeigt sich am selben Basismodell: Ein nicht-interaktiver Abruf, der die passenden Dateien in einem Zug heraussucht und daraus direkt einen Patch erzeugt, löst korrekt rund 1,3 % der Aufgaben; mit dem schrittweisen Vorgehen über das Agent-Computer Interface steigt die Lösungsrate auf 12,5 %. SWE-agent steht damit für das reaktive Vorgehen, bei dem der Agent selbst Dateien öffnet, nach passenden Stellen sucht und den nächsten Schritt anhand der Rückmeldung plant.

Xia u. a. (2024) hinterfragen mit Agentless den Nutzen einer autonomen Steuerung durch das Modell. Anstatt das System eigenständig explorieren zu lassen, geben sie einen festen, dreiphasigen Prozess vor: Zuerst wird die Fehlerstelle innerhalb der Codebase lokalisiert, anschließend ein Patch erzeugt und dieser abschließend mittels automatisierter Tests validiert. Die Lokalisierung arbeitet sich dabei von der Datei über die betroffene Klasse oder Funktion bis zur konkreten Editierstelle vor. Das Modell erledigt zwar die einzelnen Teilaufgaben, doch über die nächsten Schritte entscheidet es nicht selbst, und zu weiteren Werkzeugen greift es ebenfalls nicht. Auf SWE-bench Lite, einer Teilmenge des Benchmarks mit 300 Aufgaben, löst Agentless 32,0 % der Fälle. Der Unterschied der beiden Ansätze liegt in der Kontrolle über den Suchprozess. Während Coding-Agents während ihrer Laufzeit autonom explorieren und eigenständig entscheiden, welche Schritte sie als Nächstes unternehmen und welchen Code sie abrufen, folgt Agentless einer festen prozeduralen Steuerung. Der Spielraum des Sprachmodells ist dabei strikt auf die jeweiligen Phasen der Lokalisierung, Reparatur und Validierung begrenzt, wodurch das System den vordefinierten, sequenziellen Ablauf nicht verlassen kann.

2.2 Kontext als Erfolgsfaktor und Engpass

Wie gut ein Agent den passenden Kontext findet, lässt sich inzwischen für sich genommen messen. Li u. a. (2026) setzen mit ContextBench nicht am Endergebnis an, sondern am Abruf selbst, und werten dafür 1.136 Aufgaben aus 66 Repositories in acht Programmiersprachen aus. Ihr Befund: Agenten rufen eher zu breit ab, und aufwendigere Agentenarchitekturen verbessern die Qualität des Abrufs nur wenig. Dass eine gezielte Vorbereitung des Kontextes hilft, zeigen Suri u. a. (2026). Eine Voranalyse des Repositories vor der Bearbeitung hebt die Lösungsrate auf SWE-bench Verified, einer geprüften Teilmenge realer GitHub-Aufgaben, um rund 20 % relativ gegenüber demselben Verfahren ohne diese Voranalyse.

2.3 Graphbasierte Kontextbereitstellung

Eine strukturelle Antwort darauf fasst die Codebase nicht als lose Sammlung von Dateien auf, sondern als Graphen aus Code-Einheiten und ihren Beziehungen. Ouyang u. a. (2025) zeigen mit RepoGraph, dass ein solcher Graph sich als Zusatzmodul in bestehende Systeme einfügen lässt und deren Lösungsrate auf SWE-bench durchweg anhebt. KGCompass von Yang u. a. (2025) nimmt zusätzlich die Issues und Pull Requests eines Repositories in den Graphen auf, neben den Code-Einheiten wie Dateien, Klassen und Funktionen. Ausgehend von einer Fehlerbeschreibung führen die so entstandenen Verbindungen über mehrere Kanten hinweg zu den passenden Codestellen. Auf SWE-bench Lite löst KGCompass 58,3 % der Aufgaben, und von den Fehlern, die es lokalisiert, sind 89,7 % im Issue-Text nicht ausdrücklich genannt und nur über solche mehrstufigen Pfade im Graphen erreichbar.

2.4 Grenzen langer, unstrukturierter Kontexte (Modellseite)

Auch ein gut ausgewählter Kontext kann die Leistung mindern, sobald er unstrukturiert und umfangreich beim Modell ankommt. Der Grund liegt im Modell selbst. Liu u. a. (2023) beschreiben mit Lost-in-the-Middle, dass Modelle Angaben am Anfang und Ende einer langen Eingabe gut verwerten, in der Mitte dagegen spürbar schlechter, und das auch dann, wenn sie für lange Kontexte gebaut sind. Diese Positionsabhängigkeit betrifft nicht nur ältere Modelle. Wu u. a. (2025) führen sie theoretisch auf die Transformer-Architektur selbst zurück und reproduzieren dieselbe Verzerrung in aktuellen Sprachmodellen. Über die Position hinaus wirkt schon die schiere Länge: Du u. a. (2025) messen über mehrere Modelle hinweg, dass die Leistung mit wachsender Eingabe nachlässt, auch wenn alle nötigen Angaben enthalten sind und nichts Ablenkendes

hinzukommt.

2.5 Nachvollziehbarkeit und menschliche Beteiligung

Neben der Frage, welchen Kontext ein Agent nutzt, steht die Frage, ob sich sein Vorgehen überhaupt nachvollziehen lässt. Hutter und Pradel (2026) setzen hier mit AgentStepper an, einem interaktiven Debugger für Coding-Agents. Er macht die Trajektorie eines Agenten Schritt für Schritt sichtbar, seine Modellanfragen, Werkzeugaufrufe und Code-Änderungen, und man kann an einzelnen Stellen anhalten und eingreifen. Adressat ist der Entwickler, der den Agenten selbst prüfen und verbessern will. Das Werkzeug setzt damit nach der Ausführung an. Andere Forschungsansätze beziehen den Menschen bereits in die Phase der Kontextaufbereitung, das sogenannte Retrieval, ein. Dies geschieht etwa durch die manuelle Überprüfung abgerufener Textstellen (raggy, Romero Lauro u. a. 2025) oder durch die Unterstützung bei der Formulierung von Anfragen an Wissensgraphen (LinkQ, Li u. a. 2025; Pusch u. a. 2026). Inwiefern sich die vorliegende Arbeit von diesen und weiteren verwandten Ansätzen unterscheidet, wird im folgenden Kapitel dargelegt.

3 Abgrenzung

3.1 Einleitung

Diese Arbeit verbindet drei verschiedene Forschungsbereiche, die bisher unabhängig voneinander untersucht wurden. Der erste Bereich beschäftigt sich mit automatischem, graphbasiertem Code-Retrieval. Hier geht es um Systeme, die aus der vorhandenen Codebase einen Wissensgraphen erstellen, der dann vom KI-Modell abgefragt werden kann. Der zweite Bereich behandelt das Sichtbarmachen von Kontext-Retrieval-Pipelines für den Entwickler. Dazu gibt es verschiedene Werkzeuge, die das Retrieval für den Menschen nachvollziehbar machen. Es wird beispielsweise sichtbar gemacht, welche Textabschnitte (Chunks) bei einer RAG-Abfrage zurückgekommen sind und wie diese ausgewählt wurden. Der dritte Bereich behandelt menschengestützte Interaktion bei der Formulierung von Graph-Anfragen. Dabei hilft der Nutzer einem KI-Modell, eine korrekte Abfrage zu bauen, und prüft sie.

Die in den Abschnitten 2.3 und 2.4 dargelegten Befunde machen den strukturellen Vorteil einer graphbasierten Kontextbereitstellung deutlich. Im Unterschied zu einer flachen Übergabe, bei der relevante Code-Abschnitte unstrukturiert aneinandergereiht werden, speichert ein Wissensgraph Code-Einheiten als Knoten und ihre Abhängigkeiten

als typisierte Kanten. Ausgehend von identifizierten Einstiegspunkten lässt sich dem Graphen entlang seiner Kanten folgen und so gezielt weiterer zusammenhängender Code erschließen, ohne den gesamten Kontext laden zu müssen. Die im Code ausdrücklich vorhandene Beziehungsstruktur, zu der Aufrufketten, Importpfade und Klassenvererbungen zählen, bleibt dabei erhalten, statt vom Modell aus einer unstrukturierten Liste rekonstruiert werden zu müssen.

Diese drei Aspekte sind in der Forschung bislang getrennt voneinander untersucht worden.

3.2 Strang 1: Automatisches graphbasiertes Code-Retrieval

Es gibt eine Reihe von Systemen, die automatisches graphbasiertes Code-Retrieval durchführen. Dabei werden Code-Entitäten und Beziehungen in einem Graph gespeichert und dem Modell per Retrieval als Kontext bereitgestellt. Erstellung und Abfrage des Graphen ist dabei meist automatisiert. Das Tool RepoGraph (Ouyang u. a. 2025) erstellt automatisiert einen zeilengranularen Graphen, der den Code und seine Struktur repräsentiert. Der KI-Agent fragt den Graphen ab und bekommt für seine Anfrage einen Subgraphen mit den als relevant eingestuften Knoten als Antwort. Diese Informationen aus dem Subgraph werden dann in den Kontext des Agenten eingebunden. CodexGraph (Liu u. a. 2024) verwendet ebenfalls einen Wissensgraphen zur Repräsentation von Code. Dabei formuliert der Agent Graph-Abfragen aktiv, um seinen Kontext iterativ anzureichern. RANGER (Shah u. a. 2025) erweitert diesen Code-Graph-Ansatz auf einen repository-weiten Wissensgraphen bis auf Variablenebene. Er reichert die Knoten mit textuellen Beschreibungen und Embeddings (Vektordarstellungen von Text) an und übersetzt Nutzeranfragen zur Laufzeit in Cypher (der Abfragesprache von Neo4j, einer graphbasierten Datenbank). Codebase-Memory (Vogel u. a. 2026) geht einen ähnlichen Weg und baut seinen Code-Wissensgraphen mit tree-sitter auf, einem Parser, der Quellcode anhand seiner syntaktischen Struktur zerlegt. Bereitgestellt wird der Graph über das Model Context Protocol (MCP), worüber ein KI-Agent ihn mit strukturellen Anfragen wie Aufrufketten oder Impact-Analysen (welche Codeteile von einer Änderung betroffen sind) direkt durchsuchen kann. Von den hier genannten Systemen ähnelt dieser Ansatz dem in dieser Arbeit verwendeten Retrieval-Kern am meisten.

Kritik/Lücke: Diese Werkzeuge haben gemeinsam, dass das Retrieval vollständig automatisch läuft. Die Subgraphen werden automatisch erzeugt und gehen ohne menschliche Prüfung in den Kontext des KI-Modells bzw. -Agenten. Warum welcher Knoten für den Subgraphen ausgewählt wurde, wird dem Entwickler nicht offengelegt. Außerdem kann er vor der Generierung einer Antwort auf die gestellte Anfrage weder eingreifen noch die Auswahl der Knoten und Kanten im Subgraph verändern. Das in dieser Masterarbeit

entwickelte System setzt hier an: Es legt die Knotenauswahl dem Entwickler offen und ermöglicht ihm, sie vor der Übergabe an den Agenten interaktiv anzupassen.

3.3 Strang 2: Interaktive Inspektion von Retrieval-Pipelines

Grundidee: Die Retrieval-Stufe wird für Menschen sichtbar und prüfbar. Dieser Bereich befasst sich mit der Sichtbarmachung und Prüfung von Kontext-Retrieval-Pipelines. Die verschiedenen Werkzeuge sind: *raggy* (Romero Lauro u. a. 2025), *XGraphRAG* (Wang u. a. 2025) und *RAGAPHENE* (Fadnis u. a. 2025). *raggy* ist eine interaktive Oberfläche für Entwickler, die abgerufene Text-Chunks und Ähnlichkeits-Scores aus RAG-Anfragen sichtbar macht. *raggy* bietet dem Entwickler die Möglichkeit, Chunks manuell an- oder abzuwählen und die Retrieval-Parameter live anzupassen und zu optimieren. *XGraphRAG* überträgt diese Idee auf Graph-RAGs. Es arbeitet auf einem aus Text konstruierten Dokument-Graphen. Dabei wurde es explizit für Input-Tripel gebaut. Diese bestehen aus einer Frage, der Ground Truth (der als korrekt hinterlegten Antwort) und der Menge der Fakten (Graph-Elemente/Kanten), die die richtige Antwort stützen. Es vergleicht eine belegte, vor Ausführung bekannte, richtige Antwort mit der tatsächlichen Antwort des Graph-RAG. Es markiert beispielsweise Missing Recalls (was hätte abgerufen werden müssen, wurde aber nicht) und Unexpected Recalls (was fälschlicherweise abgerufen wurde) und verfolgt eine Antwort Schritt für Schritt durch die Retrieval-Pipeline. Es dient der Fehleranalyse der Graph-RAG-Pipeline und zeigt ihrem Entwickler, in welcher Phase eine Fehlinformation in den Subgraphen gelangte, beim Aufbau des Graphen oder beim späteren Abruf. RAGAPHENE verfolgt einen chat-basierten Annotationsansatz. Annotatoren prüfen dort die automatisch abgerufenen Passagen, verwerfen irrelevante, ergänzen fehlende und erzeugen die Antwort neu, korrigieren sie bei Bedarf von Hand und versehen sie mit Metadaten wie Fragetyp und Beantwortbarkeit. Ziel ist es, auf diese Weise Benchmark- und Evaluationsdaten für RAG-Systeme zu gewinnen.

Kritik/Lücke: Alle drei Werkzeuge, *raggy*, *RAGAPHENE* und *XGraphRAG*, haben gemein, dass sie für Entwickler oder Annotatoren gedacht sind und rückblickend arbeiten. Sie fragen, warum ein Ergebnis so zustande kam und nicht, wie sich der Kontext vorab formen lässt, auf dem ein Agent eine konkrete Aufgabe löst. Hinzu kommt ein Unterschied in der Datenstruktur: raggy und RAGAPHENE operieren auf Text-Passagen, ohne jede Graph-Traversierung. XGraphRAG arbeitet dagegen auf einem aus Text konstruierten Dokument-Graphen und nicht auf einem Code-Graphen. XGraphRAG setzt außerdem die richtige Antwort bereits voraus und eignet sich damit für die nachgelagerte Fehleranalyse, nicht für eine noch ungelöste Aufgabe. RAGAPHENE wiederum erzeugt Annotations- und Evaluationsdaten und läuft damit außerhalb der eigentlichen Aufgabe

eines Agenten. Und schließlich zielen alle drei auf allgemeine Dokument-Frage-Antwort Probleme, nicht auf Code oder Coding-Agenten.

Trotz dieser Unterschiede ist gerade dieser Strang für die vorliegende Arbeit besonders aufschlussreich. Er zeigt, dass es sich lohnt, das Retrieval nicht als Blackbox zu behandeln, sondern den Menschen das Abgerufene prüfen und korrigieren zu lassen. Bei raggy und RAGAPHENE geschieht dies auf der Ebene der einzelnen abgerufenen Elemente. Dass dieser Bedarf real ist, belegt raggy empirisch: Die Teilnehmenden der Studie verlangten von sich aus nach mehr Klarheit darüber, warum ein Element überhaupt abgerufen wurde. Genau diesen Gedanken greift die vorliegende Arbeit auf und überträgt ihn von der rückblickenden Analyse auf die prospektive Kontextauswahl, von flachen Passagen auf einen Code-Graphen und von der Entwickler- bzw. Annotationsarbeit auf die laufende Aufgabe eines Coding-Agenten.

3.4 Strang 3: Menschengestützte Interfaces für Wissensgraphen

Ein dritter Forschungsbereich bezieht den Menschen in die Formulierung der Anfrage an einen Wissensgraphen ein. LinkQ (Li u. a. 2025) ist ein sprachmodellgestütztes visuelles Interface: Über ein mehrstufiges Protokoll überführt es eine natürlichsprachige Frage in eine wohlgeformte Anfrage in SPARQL (einer standardisierten Abfragesprache für Wissensgraphen) gegen den Wissensgraphen *Wikidata*. Damit der Nutzer die Anfrage auf Korrektheit prüfen und Fehler erkennen kann, zeigt LinkQ zwei Ansichten an, eine Entity-Relation-Tabelle und einen Query-Graphen (einer visuellen Darstellung der Anfrage).

Der in dieser Masterarbeit entwickelte Ansatz ähnelt am stärksten dem interaktiven Framework von Pusch u. a. (2026). Dieses Framework reagiert auf mehrere Schwächen bestehender Graph-RAG-Systeme. Manche graphbasierte Systeme vereinfachen den Wissensgraphen zu einer bloßen Liste von Einzelfakten, sodass seine Beziehungsstruktur verloren geht und das Sprachmodell die Abhängigkeiten aus der flachen Liste erst mühsam rekonstruieren muss. Andere finden die passenden Knoten über Ähnlichkeitsvergleiche, legen aber nicht offen, warum gerade diese gewählt wurden, sodass der Abruf eine Blackbox bleibt. In beiden Fällen ist der Nutzer nur passiver Empfänger und kann nicht mehr eingreifen, sobald Suche und Antwort laufen. Genau hier setzt das Framework an. Wie LinkQ beteiligt es den Menschen an der Formulierung der Datenbankabfrage, statt ihm nur eine fertige Antwort zu liefern, und macht den Weg von der Frage zur Antwort dabei sichtbar und korrigierbar.

Dazu verteilt das Framework die Arbeit auf vier aufeinander folgende Schritte. Zuerst übersetzt ein Sprachmodell die in Alltagssprache gestellte Frage in eine passende Abfrage

in der Graph-Abfragesprache Cypher (der Generator) und führt sie auf der Graphdatenbank aus (der Executor). Danach erklärt dasselbe Modell in einfachen Worten, was die Abfrage bewirkt, und weist auf Auffälligkeiten hin, etwa eine Beziehung, die in die falsche Richtung zeigt, oder einen Knotentyp, den es im Graphen gar nicht gibt (der Explainer). Stimmt etwas nicht, muss der Nutzer nicht von vorn anfangen. Er formuliert in Alltagssprache, was geändert werden soll, und das Modell passt nur die betroffene Stelle der Abfrage an, statt eine völlig neue zu erzeugen. Diesen nachbessernden Schritt nennen die Autoren den Amender.

So kann auch jemand einen Wissensgraphen nutzen, der die zugehörige Abfragesprache nicht beherrscht, und behält zugleich Überblick und Kontrolle darüber, wie die Antwort zustande kommt. Weil alle Ergebnisse aus dem geprüften Graphen stammen, sinkt außerdem die Gefahr, dass das Sprachmodell Fakten erfindet. Getestet haben die Autoren das System an einem eigens erstellten Film-Wissensgraphen sowie an zwei echten Graphen aus der Biologie und der Mathematik.

In beiden Fällen kontrolliert der Mensch die Anfrage an den Graphen, nicht die abgerufene Kontextmenge. Er beurteilt die erzeugte Anfrage und lässt sie bei Bedarf neu generieren. Einzelne abgerufene Knoten gezielt an- oder abzuwählen, gehört, soweit beschrieben, nicht zur Interaktion. Bei LinkQ kommen drei weitere Unterschiede hinzu. Das System nutzt kein Embedding-Retrieval, sondern löst die passenden Wikidata-Bezeichner über eine Fuzzy-Suche auf und konstruiert daraus die SPARQL-Anfrage. Sein Query-Graph zeigt den Aufbau der Anfrage in Form von Basic Graph Patterns, also den Tripelmustern, aus denen sie besteht, und nicht einen abgerufenen Subgraphen, der offenlegt, warum seine Knoten abgerufen wurden. Und die Evaluation bleibt begrenzt: Sie beruht auf einer qualitativen Studie mit fünf Teilnehmenden, während die Autoren einen quantitativen Vergleich mit klassischen Ansätzen des Natural Language Processing (NLP) und mit RAG selbst als offene Aufgabe nennen. Im Unterschied dazu ermöglicht das in dieser Masterarbeit entwickelte System dem Entwickler nicht nur, die Anfrage zu steuern, sondern den abgerufenen Subgraphen selbst zu prüfen, einzelne Knoten gezielt an- oder abzuwählen und ihre Gewichtung festzulegen.

3.5 Abgrenzung und Beitrag

Die vorliegende Arbeit führt diese drei zuvor getrennten Stränge zusammen. Vom ersten übernimmt sie das graphbasierte Code-Retrieval, vom zweiten die Idee, das Abgerufene für den Menschen sichtbar und prüfbar zu machen, vom dritten die menschliche Beteiligung am Abruf. Jeder Strang für sich deckt nur einen dieser Aspekte ab. Dort, wo sie sich überschneiden, setzt das hier vorgestellte System an: Es stellt einem Coding-Agenten

seinen Kontext über ein graphbasiertes Retrieval bereit und legt zu jedem Knoten offen, warum er abgerufen wurde. Bevor der Agent beginnt, kann der Mensch diese Auswahl prüfen und formen.

Die beiden zentralen Schwächen, die Pusch u. a. (2026) an bestehenden Graph-RAG-Systemen benennen, der undurchsichtige Abruf und die Passivität des Nutzers, decken sich genau mit der Motivation dieser Arbeit und werden durch den hier gewählten Ansatz gezielt adressiert.

Das in dieser Masterarbeit entwickelte System verwendet allerdings eine andere Vorgehensweise als Pusch u. a. Im Unterschied zu ihrem Ansatz behält es das embeddingbasierte Retrieval bei. Es macht für jeden Knoten nachvollziehbar, warum und wie er abgerufen wurde: Ähnlichkeits-Score, Schwellwert, Ablehnungsgrund, eingebetteter Text und der Pfad, über den er erreicht wurde. Kuratiert wird damit nicht wie bei Pusch u. a. die Anfrage, sondern der graphbasierte Abruf selbst: Der Entwickler steuert, welche Knoten die Suche seeden, welchen Verknüpfungen sie folgt und wie tief, und in welcher Reihenfolge die Knoten gewichtet in den Kontext gehen. Geformt wird auf diese Weise die Struktur des abgerufenen Subgraphen, nicht eine flache Kontextmenge. Die von Pusch u. a. beschriebene Opazität wird auf diese Weise nicht durch den Verzicht auf Embedding-Retrieval beantwortet, sondern dadurch, dass das Embedding-Retrieval transparent und für den Entwickler modifizierbar gemacht wird.

Das an anderen Systemen kritisierte Einebnen der Graphstruktur (Pusch u. a. 2026) greift beim hier vorgestellten System nicht. Der kuratierte Subgraph wird dem Agenten nicht als undifferenzierter Text übergeben, sondern als strukturierte, mit Herkunftsangaben angereicherte Darstellung. Die Knoten sind nach Traversierungsdistanz gegliedert, von den Einstiegspunkten über die direkten Abhängigkeiten bis zum transitiven Kontext, und jeder Knoten trägt den Pfad, über den er erreicht wurde, mit explizit typisierten Kanten wie CALLS oder IMPORTS. Im interaktiv bestätigten Fall kommt die vollständige gerichtete Kantenmenge zwischen den ausgewählten Knoten hinzu. Der Code selbst wird über Datei- und Zeilenreferenzen angesprochen und nicht vollständig in den Kontext übertragen, was zugleich das von Pusch u. a. beklagte Aufblähen des Kontextfensters vermeidet. Die explizite Struktur des Graphen bleibt damit auch in der Übergabe erhalten, statt vom Agenten aus einer flachen Aufzählung rekonstruiert werden zu müssen.

Der Retrieval-Kern des hier vorgestellten Systems steht den automatischen Code-Graph-Systemen aus dem ersten Strang nahe, am ehesten dem von Codebase-Memory. Der Unterschied liegt an anderer Stelle. RepoGraph, CodexGraph, RANGER und Codebase-Memory reichen den erzeugten Subgraphen unmittelbar an das Modell weiter. Das in dieser Masterarbeit entwickelte System schiebt davor einen Schritt, der die Auswahl

sichtbar und veränderbar macht. Der abgerufene Subgraph wird dem Menschen gezeigt, zu jedem Knoten wird offengelegt, warum er abgerufen wurde, und er kann die Auswahl anpassen, bevor der Agent auf ihr arbeitet. Neu ist damit nicht, wie der Graph entsteht, sondern eine Transparenz- und Kontrollschicht über einem im Kern vergleichbaren Abruf.

Auch das Verfahren XGraphRAG legt die Graphenstruktur offen und ermöglicht Modifikation der Struktur, scheidet aber für eine offene Aufgabe aus, da XGraphRAG die Kenntnis der richtigen Antwort voraussetzt und daher nur eine retrospektive Analyse erlaubt. In dem in dieser Masterarbeit entwickelten Verfahren setzt die Kuratierung des Kontext-Graphen an, bevor die Antwort feststeht. Der Kontext und seine strukturellen Verbindungen werden vor der Übergabe an den Agenten geformt, um durch eine verbesserte, strukturierte Kontextübergabe eine effizientere Lösung von Programmieraufgaben zu ermöglichen.

Das Forschungsfeld der Coding-Agents und graphbasierten Kontextbereitstellung entwickelt sich derzeit sehr dynamisch. Neue Systeme und Ansätze werden in kurzen Abständen veröffentlicht. Der in dieser Arbeit dargestellte Stand der Forschung sowie die vorgenommene Abgrenzung spiegeln den Zeitpunkt wider, zu dem das Thema dieser Masterarbeit festgelegt wurde. Später erschienene Arbeiten konnten nicht mehr vollständig berücksichtigt werden.

Literaturverzeichnis

Anthropic (2024). *Model Context Protocol: An Open Standard for LLM Tool Integration*. Veröffentlicht November 2024. URL: https://modelcontextprotocol.io (besucht am 15. 07. 2026).

Du, Yufeng u. a. (2025). *Context Length Alone Hurts LLM Performance Despite Perfect Retrieval*. arXiv: 2510.05381 [cs.CL]. URL: https://arxiv.org/abs/2510.05381.

Fadnis, Kshitij, Sara Rosenthal, Maeda Hanafi, Yannis Katsis und Marina Danilevsky (2025). *RAGAPHENE: A RAG Annotation Platform with Human Enhancements and Edits*. arXiv: 2508.19272 [cs.CL]. URL: https://arxiv.org/abs/2508.19272.

Hutter, Robert und Michael Pradel (2026). *AgentStepper: Interactive Debugging of Software Development Agents*. arXiv: 2602.06593 [cs.SE]. URL: https://arxiv.org/abs/2602.06593.

Jimenez, Carlos E. u. a. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv: 2310.06770 [cs.CL]. URL: https://arxiv.org/abs/2310.06770.

Li, Han u. a. (2026). *ContextBench: A Benchmark for Context Retrieval in Coding Agents*. arXiv: 2602.05892 [cs.LG]. URL: https://arxiv.org/abs/2602.05892.

Li, Harry, Gabriel Appleby und Ashley Suh (2025). *LinkQ: An LLM-Assisted Visual Interface for Knowledge Graph Question-Answering*. DOI: 10.1109/VIS55277.2024.00031. arXiv: 2406.06621 [cs.CL]. URL: https://arxiv.org/abs/2406.06621.

Liu, Nelson F. u. a. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv: 2307.03172 [cs.CL]. URL: https://arxiv.org/abs/2307.03172.

Liu, Xiangyan u. a. (2024). *CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases*. arXiv: 2408.03910 [cs.SE]. URL: https://arxiv.org/abs/2408.03910.

Lulla, Jai Lal, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes und Christoph Treude (2026). *On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents*. arXiv: 2601.20404 [cs.SE]. URL: https://arxiv.org/abs/2601.20404.

Ouyang, Siru u. a. (2025). *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. arXiv: 2410.14684 [cs.SE]. URL: https://arxiv.org/abs/2410.14684.

Pusch, Larissa, Alexandre Courtiol und Tim Conrad (2026). *A Human-in-the-Loop, LLM-Centered Architecture for Knowledge-Graph Question Answering*. arXiv: 2602.05512 [cs.CL]. URL: https://arxiv.org/abs/2602.05512.

Romero Lauro, Quentin, Shreya Shankar, Sepanta Zeighami und Aditya Parameswaran (2025). *RAG Without the Lag: Interactive Debugging for Retrieval-Augmented Gene-*

ration Pipelines. arXiv: 2504.13587 [cs.HC]. URL: https://arxiv.org/abs/2504.13587.

Shah, Pratik, Rajat Ghosh, Aryan Singhal und Debojyoti Dutta (2025). *RANGER – Repository-Level Agent for Graph-Enhanced Retrieval*. arXiv: 2509.25257 [cs.SE]. URL: https://arxiv.org/abs/2509.25257.

Suri, Manan u. a. (2026). *CodeScout: Contextual Problem Statement Enhancement for Software Agents*. arXiv: 2603.05744 [cs.CL]. URL: https://arxiv.org/abs/2603.05744.

Vogel, Martin, Falk Meyer-Eschenbach, Severin Kohler, Elias Grünewald und Felix Balzer (2026). *Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP*. arXiv: 2603.27277 [cs.SE]. URL: https://arxiv.org/abs/2603.27277.

Wang, Ke u. a. (2025). *XGraphRAG: Interactive Visual Analysis for Graph-based Retrieval-Augmented Generation*. arXiv: 2506.13782 [cs.IR]. URL: https://arxiv.org/abs/2506.13782.

Wu, Xinyi, Yifei Wang, Stefanie Jegelka und Ali Jadbabaie (2025). *On the Emergence of Position Bias in Transformers*. arXiv: 2502.01951 [cs.LG]. URL: https://arxiv.org/abs/2502.01951.

Xia, Chunqiu Steven, Yinlin Deng, Soren Dunn und Lingming Zhang (2024). *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv: 2407.01489 [cs.SE]. URL: https://arxiv.org/abs/2407.01489.

Yang, Boyang u. a. (2025). *Enhancing repository-level software repair via repository-aware knowledge graphs*. arXiv: 2503.21710 [cs.SE]. URL: https://arxiv.org/abs/2503.21710.

Yang, John u. a. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv: 2405.15793 [cs.SE]. URL: https://arxiv.org/abs/2405.15793.